



Pauta Tarea 1

INDICACIONES

- Esta Pauta muestra una forma de solución a los problemas.
- Estos problemas tienen varias maneras de resolverse. Todos los códigos son distintos.
- Se evalúa aprendizaje de conceptos y comprensión del problema.

Problemas

1. Clasificación de número entero.

Enunciado

- Escriba un programa en Python que solicite al usuario ingresar un número entero.
- El programa debe analizar el valor ingresado y determinar si corresponde a un número par o impar. Para lograr el objetivo, divida el número ingresado por 2, y evalúe el resto utilizando la operación módulo %
- Para realizar la clasificación, utilice únicamente estructuras condicionales (if, elif, else).
- Finalmente, el programa debe mostrar un mensaje indicando si el número ingresado es un par o impar.

Solución.

Idea

El programa debe solicitar al usuario un número entero. Luego debe verificar si el número es divisible por 2. Para realizar esta verificación se utiliza la operación módulo %. El operador módulo devuelve el resto de una división.

Por lo tanto:

- Si el resto de dividir el número por 2 es igual a 0, el número es **par**.
- Si el resto es diferente de 0, el número es **impar**.

Esta clasificación se puede realizar utilizando estructuras condicionales.

Implementación

```
1 numero = int(input("Ingrese un número entero: "))
2
3 if numero % 2 == 0:
4     print("El número ingresado es par.")
5 else:
6     print("El número ingresado es impar.")
```

Explicación del algoritmo

Primero se solicita al usuario un número entero:

```
1 numero = int(input("Ingrese un número entero: "))
```

La función `int()` convierte el valor ingresado a un número entero.

Luego se evalúa la condición utilizando el operador módulo:

```
1 if numero % 2 == 0:
```

Aquí ocurre lo siguiente:

- `numero % 2` calcula el resto de dividir el número por 2.
- Si el resto es 0, el número es divisible por 2.
- Todo número divisible por 2 es un número par.

Ejemplo:

- Si el usuario ingresa 8 entonces $8 \% 2 = 0$. Por lo tanto el número es par.
- Si el usuario ingresa 7 entonces $7 \% 2 = 1$. Por lo tanto el número es impar.

Dependiendo del resultado de la condición, el programa imprime el mensaje correspondiente mediante `print()`.

Resultado

El programa permite clasificar correctamente si el número ingresado es par o impar utilizando la operación módulo y estructuras condicionales.

2. Iteradores para sumatoria y pitatoria.

Enunciado

- Resuelva la siguiente operación:

$$\text{Resultado} = \sum_{x=1}^{n_1} \pi \frac{x}{12} + \prod_{x=1}^{n_2} x^2.$$

- Indique al usuario que ingrese dos valores de entrada para n_1 y n_2 , donde n_1 corresponde a la sumatoria y n_2 a la pitatoria.
- Aplique dos iteradores independientes para calcular primero la sumatoria y luego la pitatoria.
- Finalmente, imprima la operación correspondiente.
- Analice qué ocurre si $n_1 = 88$ y $n_2 = 77$.
- Deje como comentario al final del código:
 - el resultado de la sumatoria,
 - el resultado de la pitatoria,
 - el resultado final.
- **Bonus:** escriba el mismo programa utilizando dos funciones por separado. Luego sume los resultados de estas funciones en una variable e imprímala utilizando los mismos valores anteriores. Determine si se obtiene el mismo resultado.

Solución.

Observación previa. Para esta versión del ejercicio, la operación pedida es

$$\sum_{x=1}^{n_1} \pi \frac{x}{12} + \prod_{x=1}^{n_2} x^2.$$

Es decir, una *sumatoria lineal multiplicada por $\pi/12$* y una *productoria de cuadrados*.

Idea del algoritmo

Se necesitan dos procesos separados:

- una variable acumuladora para la sumatoria, que debe comenzar en 0, porque 0 es el elemento neutro de la suma;
- una variable acumuladora para la productoria, que debe comenzar en 1, porque 1 es el elemento neutro de la multiplicación.

Luego:

- un primer iterador recorre los valores $x = 1, 2, \dots, n_1$ y va sumando el término $\pi \frac{x}{12}$;
- un segundo iterador recorre los valores $x = 1, 2, \dots, n_2$ y va multiplicando x^2 ;
- al final, el resultado pedido es la suma de ambos acumulados, dejando primero la sumatoria y luego la productoria.

Implementación con dos iteradores independientes

```
1 import math
2
3 # Solicitar datos de entrada
4 n1 = int(input("Ingrese el valor de n1: "))
5 n2 = int(input("Ingrese el valor de n2: "))
6
7 # Acumulador de la sumatoria
8 sumatoria = 0
9
10 # Primer iterador: calcula \sum_{x=1}^{n1} pi*x/12
11 for x in range(1, n1 + 1):
12     sumatoria += math.pi * x / 12
13
14 # Acumulador de la productoria
15 productoria = 1
16
17 # Segundo iterador: calcula \prod_{x=1}^{n2} x^2
18 for x in range(1, n2 + 1):
19     productoria *= x ** 2
20
21 # Resultado final
22 resultado = sumatoria + productoria
23
24 # Mostrar resultados
25 print(f"La sumatoria es: {sumatoria:.3f}")
26 print(f"La productoria es: {productoria:.3e}")
27 print(f"El resultado final es: {resultado:.3e}")
28
29 # Para n1 = 88 y n2 = 77:
```

```

30 # sumatoria = 1025.206
31 # productoria = 2.108e+226
32 # resultado final = 2.108e+226

```

Qué hace cada parte del código

Entrada de datos. Las líneas

```
n1 = int(input(...)), n2 = int(input(...))
```

piden al usuario dos enteros, porque ambos límites de iteración deben ser números enteros positivos.

Sumatoria. Se define

```
sumatoria = 0
```

porque ahora queremos ir agregando términos. El primer ciclo recorre los enteros desde 1 hasta n_1 , inclusive. En cada iteración se suma

```
math.pi * x / 12,
```

es decir, el término $\pi x/12$ correspondiente al valor actual de x .

Productoria. Se define

```
productoria = 1
```

porque si iniciáramos en 0, cualquier multiplicación posterior seguiría dando 0. Después, el segundo ciclo

```
for x in range(1, n2 + 1)
```

recorre los enteros desde 1 hasta n_2 , inclusive. En cada vuelta del ciclo se actualiza el acumulador mediante

```
productoria *= x ** 2,
```

que significa multiplicar sucesivamente por los cuadrados x^2 .

Resultado final. Una vez terminados ambos ciclos, ya se tienen calculadas las dos partes de la expresión. Por eso basta escribir

```
resultado = sumatoria + productoria.
```

Análisis para $n_1 = 88$ y $n_2 = 77$

Si se evalúa el programa con esos valores, se obtiene:

$$\sum_{x=1}^{88} \pi \frac{x}{12} \approx 1025,206,$$

mientras que la productoria vale aproximadamente

$$\prod_{x=1}^{77} x^2 \approx 2,107 \times 10^{226}.$$

Entonces el resultado final es

$$\text{Resultado} \approx 2,107 \times 10^{226}.$$

Interpretación. Aquí ocurre algo interesante: la productoria es tan grande en comparación con la sumatoria que, al sumarlas en aritmética de punto flotante, el aporte de 1025,206 no cambia el valor mostrado por pantalla. En la práctica computacional, el resultado final coincide con la productoria a las cifras que puede representar el tipo numérico usado al imprimir con tres decimales en notación científica. La sumatoria no desaparece matemáticamente, pero sí queda completamente eclipsada numéricamente.

Bonus: versión con dos funciones

Ahora conviene separar cada cálculo en su propia función. Esto mejora la organización del programa porque:

- cada función tiene una tarea clara y única;
- el código principal queda más limpio;
- si más adelante se quiere reutilizar una de las operaciones, no es necesario reescribirla.

```
1 import math
2
3 # Función para calcular la sumatoria
4 def calcular_sumatoria(n1):
5     sumatoria = 0
6     for x in range(1, n1 + 1):
7         sumatoria += math.pi * x / 12
8     return sumatoria
9
10 # Función para calcular la productoria
11 def calcular_productoria(n2):
12     productoria = 1
13     for x in range(1, n2 + 1):
14         productoria *= x ** 2
15     return productoria
16
17 # Solicitar datos
18 n1 = int(input("Ingrese el valor de n1: "))
19 n2 = int(input("Ingrese el valor de n2: "))
20
21 # Llamar a las funciones
22 sumatoria = calcular_sumatoria(n1)
23 productoria = calcular_productoria(n2)
24
25 # Sumar los resultados
26 resultado = sumatoria + productoria
27
28 # Imprimir resultados
29 print(f"La sumatoria es: {sumatoria:.3f}")
30 print(f"La productoria es: {productoria:.3e}")
31 print(f"El resultado final es: {resultado:.3e}")
32
33 # Para n1 = 88 y n2 = 77:
34 # sumatoria = 1025.206
35 # productoria = 2.108e+226
36 # resultado final = 2.108e+226
```

¿Se obtiene el mismo resultado?

Sí. Se obtiene el mismo resultado numérico. La razón es que las funciones no cambian la operación matemática; solamente encapsulan el cálculo. La sumatoria sigue siendo la misma, la productoria sigue siendo la misma y, por tanto, su suma también.

Conclusión. La estrategia correcta consiste en usar dos acumuladores y dos iteradores independientes. Para $n_1 = 88$ y $n_2 = 77$, la sumatoria vale aproximadamente 1025,206, pero la productoria domina por completo el resultado final. La versión con funciones separadas entrega el mismo valor, aunque con una estructura de programa más ordenada y reutilizable.